

Projeto Final de Programação Avançada



Autores

David Martins Correia – 2023140410

Tiago Gouveia Filipe – 2019112767



**Instituto Superior
de Engenharia**

Politécnico de Coimbra

Coimbra, 14/02/2025

ÍNDICE:

Introdução	2
Principais Padrões de Design Utilizados	4
Funcionalidades Implementadas	5
Conclusão	7
Referências	Erro! Marcador não definido.

INTRODUÇÃO

O presente relatório documenta o desenvolvimento de uma aplicação completa de Xadrez, concebida como projeto final da unidade curricular de Programação Avançada no ISEC. A aplicação foi construída em Java, com a interface gráfica em JavaFX e lógica de jogo estruturada com base no padrão MVC (Model-View-Controller).

O foco do projeto foi implementar de forma fiel às regras oficiais do xadrez um sistema capaz de oferecer:

Jogabilidade completa (movimentos, xeque-mate, empate);

Funcionalidades de controlo como Undo/Redo, guardar/carregar jogos;

Mecanismos especiais como Roque, Promoção e En Passant;

Uma experiência didática através do Modo Aprendizagem com feedback visual e áudio

ARQUITETURA E PRINCIPAIS DECISÕES DE IMPLEMENTAÇÃO

A arquitetura é baseada no padrão MVC, com clara separação entre:

Modelo (model): Lida com toda a lógica do jogo incluindo a característica de undo e redo (ex: Board, Command, Chessgame), onde também está incluído o controlador, facade, ChessGameManager que faz a ligação entre a lógica de jogo e a interface gráfica.

Interface Gráfica (UI): Interface desenvolvida com JavaFX, implementando tabuleiro, menus, janelas modais (nomes, logs) e ChoiceDialog (promoção do peão).

Esta abordagem promove uma clara separação entre a lógica do jogo, a interface gráfica e a camada de controlo, o que facilita a manutenção do código. Permite uma maior escalabilidade da aplicação e favorece a reutilização de componentes em diferentes contextos.

Além disso, foi implementado um sistema de comandos (Command Pattern) para gerir as ações e garantir Undo/Redo, inclusive para movimentos compostos como o Roque.

PRINCIPAIS PADRÕES DE DESIGN UTILIZADOS

Command

O padrão Command foi utilizado para encapsular operações de jogada (movimentos de peças, incluindo promoção e roque).

Cada comando guarda informações essenciais da jogada, permitindo funcionalidades de undo/redo.

Observer

Descrição: Permite que objetos sejam notificados sobre alterações de estado de outros objetos.

Aplicação: ChessGameManager usa PropertyChangedSupport para notificar a interface (ChessBoardCanvas, etc.) sempre que há mudanças no tabuleiro ou jogador.

Garante atualização automática e reativa da interface gráfica.

Singleton

Garante que uma classe tenha apenas uma instância global usando para tal um construtor privado e o método getInstance() para o uso da classe. A classe ModelLog segue o padrão Singleton para manter um histórico global dos logs e mensagens do sistema.

Facade

Oferece uma interface simplificada para um subsistema complexo a sua aplicação no ChessGameManager age como fachada do modelo, fornecendo métodos como movePiece, importar jogos, redo, etc., escondendo detalhes internos da lógica do jogo (ChessGame), tornando-se uma classe mais simples e adaptável.

Factory

Aplicado na criação de instâncias de peças (Piece) como King, Queen, Rook, etc., a partir de dados genéricos (como caracteres ou strings). Este padrão simplifica a reconstrução do tabuleiro ao importar jogos, inicializar o jogo ou promover peões, centralizando a lógica de criação num único ponto.

CLASSES UTILIZADAS

Piece	Classe abstrata que implementa a logica comum entre as várias peças
Pawn, Bishop, Knight, etc.	Subclasses de Piece com os movimentos específicos de cada tipo de peça
Pecas	Enum Pecas funciona como uma Factory para criar os objetos das diferentes peças de xadrez.
Board	Representa o tabuleiro e faz a ligação entre as peças e os seus movimentos
ChessGame	Contem o estado do jogo e a logica de movimentos e regras de xadrez
ChessGameManager	Classe facade entre UI e logica de jogo, responsável pelas alterações gráficas
ChessGameSerialization	Classe responsável por guardar e carregar o estado do jogo através de serialização de objetos
ModelLog	Classe Singleton utilizada para os registos de movimentos do jogo
ICommand	Interdace utilizada no padrao Command para encapsular e restaurar o estado do jogo
AbstractCommand	Classe Abstrata de implementação da interface ICommand
CommandManager	Implementa o padrão Command para gerir a execução, undo e redo
MovePieceCommand	Implementa a logica de undo, redoo e execução de acordo com as regras do jogo

ChessBoardCanvas	Classe responsável por desenhar o tabuleiro e interação de jogo com o utilizador
ImageManager	Classe responsável por carregar e fornecer imagens usadas na interface gráfica
SoundManager	Classe responsável por carregar e tocar os sons de movimentação
CenterPane	Classe responsável pelo painel principal de jogo
InfoPane	Classe responsável por apresentar os nomes dos jogadores e o estado do jogador a jogar
JanelaInfo	Classe responsável por apresentar os logs ao utilizador
RootPane	Classe responsável pela interação do utilizador com as funcionalidades do menu

FUNCIONALIDADES IMPLEMENTADAS

O projeto implementa integralmente todas as funcionalidades definidas no enunciado. A criação de um novo jogo com nomes dos jogadores, as operações de guardar e carregar jogos utilizam serialização, suportadas pela classe ChessGameSerialization. A importação e exportação de jogos parciais em formato específico também estão disponíveis, recorrendo à mesma interface de nomes. As regras do xadrez foram implementadas na totalidade na classe ChessGame, incluindo jogadas especiais como roque, promoção e en passant. A deteção de fim de jogo contempla situações de xeque-mate e empate. O modo de aprendizagem permite desfazer e refazer jogadas, com suporte visual através da Learning Mode a indicar as possíveis opções de jogadas. Por fim, os movimentos são acompanhados por uma descrição sonora gerida pelo SoundManager, com opção de ativar ou desativar o som.

CONCLUSÃO

O desenvolvimento deste projeto permitiu aplicar, de forma prática, conceitos fundamentais da programação orientada a objetos e da engenharia de software, destacando-se a utilização de padrões de design como MVC, Command e Factory. A separação clara entre a lógica de jogo, interface gráfica e camada de controlo resultou numa arquitetura modular, no entanto nem sempre fácil de separar.

Ao longo do projeto, foi possível implementar com sucesso as regras do xadrez, incluindo movimentos especiais como roque, promoção e en passant, bem como funcionalidades adicionais como histórico de jogadas (undo/redo), guardar e carregar jogos.

Este trabalho reforçou a importância de boas práticas de design e organização do código, proporcionando uma base sólida para futuros desenvolvimentos ou adaptações.



**Instituto Superior
de Engenharia**

Politécnico de Coimbra